RV College of Engineering®, Bengaluru – 59

Department of Information Science and Engineering

DATABASE MANAGEMENT SYSTEMS (CD252IA)

Software Requirement Specification

TITLE	**CHESS TOURNAMENT MANAGEMENT AND PLAYER PERFORMANCE ANALYTICS SYSTEM**	
TEAM	1RV25IS401	GANGADHAR
	1RV25IS406	NITHISH S

1. Hardware Requirement Specification

The application is built to run on ordinary desktop or laptop hardware; no specialised hardware is required. The following minimum configuration is recommended for development and normal use.

Component	Requirement
Processor	Intel Core i3 / AMD equivalent or above
RAM	8 GB or higher
Storage	10 GB of free disk space
Display	1366 × 768 resolution or higher
Network	Internet / LAN connectivity

2. Software Requirement Specification

Frontend

- React.js - Version 19.3.0
- Vite - Version 8.1.0
- Tailwind CSS - Version 4.3.3
- Recharts - Version 3.10.1

Backend

- Node.js - Version 24.18.0 LTS
- Express.js - Version 5.1.0

Database

- PostgreSQL - Version 17.6

Development Tools

- Visual Studio Code - Version 1.105.0
- Git - Version 2.51.0 and GitHub - web-based version control platform
- Postman - Version 11.66.0
- pgAdmin 4 - Version 9.18

Supported Browsers

- Google Chrome - Version 140 or later
- Microsoft Edge - Version 140 or later
- Mozilla Firefox - Version 143 or later

3. Functional Requirements

This section specifies the functional behaviour expected from the system, organised by module.

1. **Player Management**: The administrator should be able to add a new player, update an existing player's details, search for a player, view a player's profile, and see that player's current rating reflected accurately at all times.
2. **Tournament Management**: The administrator should be able to create a new tournament, edit its details, set its start and end dates, choose its format (round-robin, knockout, or Swiss), view existing tournaments, and update a tournament's status as it moves from upcoming to ongoing to completed.
3. **Player Registration**: Players should be able to register for a tournament. The system should prevent the same player from being registered twice for one event, and each participant's registration details should be retained for the duration of the tournament.

4. **Round Management**: The system should support creating rounds within a tournament, numbering them in sequence, tracking whether a round is scheduled, in progress, or completed, and linking each round to the games played within it.
5. **Game Management**: The system should be able to pair players for a match, assign them to a specific game and board number, associate that game with the correct round, and store the relevant game details - start time, end time, and status.
6. **Result Management**: Authorised users should be able to record the outcome of a game as a win for either player or a draw, and this result should immediately feed into the standings and rating calculations.
7. **Tournament Standings**: The system should automatically keep standings up to date, tracking games played, wins, draws, losses, tournament points, and each player's current rank within the tournament.
8. **Elo-Style Rating Updates**: Whenever a result is recorded, the system should retrieve both players' current ratings, calculate each player's expected score based on the rating difference, compare that to the actual outcome, work out the resulting rating change, update each player's rating, and log the change to their rating history.
9. **Upset Detection**: The system should compare the ratings of the two players in a game and flag the result as an upset whenever a meaningfully lower-rated player defeats a higher-rated opponent.
10. **Player Performance Analytics**: For any player, the system should be able to show their win rate, draw rate, loss rate, overall rating change, average opponent rating, total games played, tournament history, and rating progression over time.
11. **Tournament Analytics**: For any tournament, the system should generate a leaderboard, player rankings, total games played, win/draw statistics, rating changes across the field, a list of upset victories, and general tournament performance statistics.
12. **Dashboard**: The system should present graphical summaries covering player rating trends, tournament rankings, match statistics, individual player performance, and upset statistics, so that this information is easy to interpret at a glance.

4. Non-Functional Requirements

Performance

Player searches, tournament queries, result processing, and analytics calculations should all return results without noticeable delay, even as the volume of stored data grows.

- **Security**

  Administrative functions should require proper authentication and authorisation, and all user input should be validated before it is processed or stored.

- **Reliability**

  Tournament, result, rating, and standings data should remain accurate and consistent, even if multiple results are being recorded around the same time.

- **Data Integrity**

  Primary keys, foreign keys, unique constraints, check constraints, and transactions should be used throughout the schema to prevent invalid or inconsistent data from being written.

- **Scalability**

  The system should continue to perform reliably as the number of players, tournaments, rounds, and games grows over time.

- **Usability**

  Tournament organisers should be able to manage events and read analytics without needing any special training or technical background.

- **Maintainability**

  The frontend, backend, and database layers should be kept reasonably decoupled and modular, so that any one of them can be updated or extended without disrupting the others.